

Full Codebase Audit & Remediation Brief

Prepared for the development team. Covers over-engineering, security, logic flaws, performance, indexing, technology choices, and macro-architecture drift — with concrete solutions and a phased fix order.

Product	ChamberQ (multi-tenant doctor chamber SaaS for Bangladesh)
Stack	Laravel 12 · Filament 4 · Livewire 3 · stancl/tenancy 3 · MySQL (prod) / SQLite (local)
Audit date	2026-08-10 / 2026-08-11
Method	Read-only code review against the live repository. Highest-severity items were line-verified against source. Not a live penetration test.
Audience	Developer / technical lead implementing fixes

Executive summary — what matters most

#	Severity	Headline
1	CRITICAL	Backup/restore can overwrite other chambers and Super Admin; wipe commits before import (chamber can be left empty); ZIP path traversal; clinical media not backed up.
2	HIGH	Medicine/condition learning never runs on real Complete-visit path; double-tap Call next skips a patient; Cancel session cancels mid-consult patient.
3	HIGH	Session Secure cookie unset; trustProxies *; no app caching; TV/queue/consult poll every 2–3s at 4–10 queries each.
4	TECH DEBT	No queue worker (afterResponse SMS); Referer-based tenancy for Livewire; Filament fi-* CSS coupling; CI PHP 8.4 vs prod 8.3.

Do not “simplify” these — they are solid: serial allocation locking + unique index; SMS wallet atomic debit; monthly commission idempotency; clinical media private disk + stream-only; TenantScope fail-closed; prescription short token; GsmText choke point; DateOnly cast.

Contents

1. How to use this brief
2. Critical & high — security and data integrity
3. Critical & high — business logic and edge cases
4. Security (medium / lower)
5. Performance, polling, and database indexes
6. Over-engineering and structural mess
7. Technology choices: shiny vs battle-tested
8. Macro-architecture drift
9. What is correctly built (leave alone)
10. Phased remediation plan (recommended order)
11. Verification checklist for the developer
12. Owner constraints the developer must not violate

1. How to use this brief

This document combines two audits: (A) over-engineering / structural complexity, and (B) a deep pass covering security, logic flaws, dependency ordering, performance, indexing, and technology risk. Findings were verified against the repository. Highest-severity items were re-read line-by-line before inclusion.

Fix in the phase order in Section 10. Phase 1 (backup/restore) is more urgent than polish. Each phase should end with a green test suite and `php artisan app:production-check` against a MySQL-shaped environment — SQLite hides foreign-key ordering failures.

Real-life framing: think of a chamber that keeps two reception desks for one door, polls the waiting-room TV every two seconds even when nothing changed, and has a “restore from backup” button that can put Clinic A's folders into Clinic B's cabinet. That is the shape of the problems below.

2. Critical & high — security and data integrity

The backup/restore feature is currently more dangerous than having no restore. Disable tenant-facing restore until Phase 1 fixes land.

S1. Tenant restore can overwrite other chambers and Super Admin

Critical

Where: `app/Services/DataImportService.php` (~222–226, 238–242); triggered from `app/Filament/TenantAdmin/Pages/DataBackup.php`

Problem: Import uses `DB::table()->upsert(..., [$primaryKey], ...)` on global id, then force-sets `tenant_id` to the restoring chamber. Eloquent `BelongsToTenant` is bypassed. All chambers share one database.

Failure scenario: Chamber A admin restores a crafted ZIP whose `users.csv` / `chambers.csv` reuse IDs that belong to Chamber B or to the platform Super Admin (`tenant_id` null). Upsert rewrites those rows and assigns them to A — account takeover or destroying the Super Admin login.

Solution: Never upsert on bare global PKs across a shared DB. Remap IDs on import (insert new keys + translate FKs), OR refuse rows whose existing `tenant_id` $\neq$ target; for users, refuse IDs with `tenant_id` IS NULL. Prefer insert-only with remapping.

S2. Restore wipe commits, then import can fail — chamber left empty

Critical

Where: `DataImportService.php` wipe (~133–151) vs import loop (~82–101); `BackupTableMap.php` `TENANT_TABLES` order (live_sessions before bookings)

Problem: Wipe runs in its own transaction and commits. Import runs table-by-table outside that transaction. `live_sessions` is listed before bookings, but `live_sessions.current_booking_id` FKs to bookings with no onDelete rule.

Failure scenario: Admin restore (replace) on a live chamber with an active queue: wipe succeeds; import dies on MySQL FK (or wipe fails mid-way). Chamber is empty/partial with no automatic rollback. Existing tests only restore patients/bookings without a pointed live session.

Solution: Single DB transaction for wipe+import. Null `current_booking_id` before wipe. Import `live_sessions` after bookings (or import with null pointer, then second pass). Add MySQL tests for active queue + mid-import failure rollback.

S3. Zip Slip: backup ZIP extract has no path validation

High

Where: `DataImportService::extractZip` — `$zip->extractTo($directory)`

Problem: Extracts the entire archive with no check that entry names stay under the temp directory.

Failure scenario: Malicious ZIP with entries like `../../.env` or `../../public/shell.php` writes outside `storage/app/backup-import/...` when the process user can write those paths. Tenant admin restore is enough.

Solution: Iterate entries; reject `..` and absolute paths; extract only basename files matching the allowlist (`manifest.json`, `*.csv`).

S4. CSV backup does not include clinical media — false sense of DR

High

Where: BackupTableMap : : TENANT_TABLES; VisitMediaService local disk paths

Problem: Table CSV ZIP has no filesystem step. visit_records.voice_path / photo_path restore as strings pointing at missing files.

Failure scenario: After a “successful” restore, voice notes and prescription photos are gone. Staff believe disaster recovery worked.

Solution: Add mysqldump (or spatie/laravel-backup) plus visit-audio/ and visit-photos/, written off-server. Relabel CSV export as data portability, not disaster recovery.

S5. prescription_items import can attach to another tenant's prescription

High

Where: BackupTableMap : : tableHasTenantColumn (prescription_items omitted); DataImportService normalizeImportedRow

Problem: prescription_items has only prescription_id (no tenant_id). Import does not rewrite or validate that FK.

Failure scenario: Crafted prescription_items.csv with Tenant B's prescription_id injects medicines into B's prescriptions while restoring as Tenant A.

Solution: On import, resolve each prescription_id to a prescription owned by the target tenant_id, or add tenant_id + composite FK.

S6. Session cookie Secure unset; encrypt false

High

Where: config/session.php secure=>env('SESSION_SECURE_COOKIE') with no default; encrypt default false; ProductionReadiness only warns

Problem: Missing env leaves Secure falsey — cookie can travel over cleartext HTTP. Combined with the deliberate one-year session lifetime (owner-locked — do NOT shorten), a stolen cookie is a year-long login.

Failure scenario: Any non-HTTPS hop (misconfigured proxy, captive portal, shared clinic Wi-Fi) → session theft → full chamber admin/doctor access.

Solution: Set SESSION_SECURE_COOKIE=true and SESSION_ENCRYPT=true in production and .env.example. Promote Secure cookie to a ProductionReadiness blocker when APP_ENV=production. Do not change SESSION_LIFETIME / AUTH_PASSWORD_TIMEOUT (locked).

S7. trustProxies(at: '*') trusts any forwarded headers

High

Where: bootstrap/app.php

Problem: Every request's X-Forwarded-* is trusted so absolute URLs work behind nginx — but any client that can hit PHP directly can spoof Host/Proto.

Failure scenario: Direct-to-PHP clients poison ticket/SMS/prescription absolute URLs (phishing) or confuse HTTPS detection.

Solution: Trust only the real reverse-proxy IP(s), not *.

3. Critical & high — business logic and edge cases

L1. Medicine/condition “learning” never runs on real Complete-visit

High

Where: CompleteBookingWithVisitNotes.php (~50–86); VisitRecordService.php (~86–88)

Problem: Usage is recorded only if booking status === 'completed'. The completion helper saves notes first (status still in_chamber), then completes the booking. Tests mark completed before save — so they pass and miss production order.

Failure scenario: Doctor completes Fatima with NAPA + gastritis. Notes save while in_chamber; status then flips to completed. condition_usages / medicine_usages stay empty. Ranked picker never learns.

Solution: Complete booking first, then save notes; OR call recordUsagesFromSubmission after status flip; OR pass \$forceUsage=true from completion helpers. Add a test that goes through CompleteBookingWithVisitNotes.

L2. Double-tap “Call next patient” skips a waiting patient

High

Where: LiveSessionService::callNextPatient / advanceQueue; Live Queue blade button

Problem: advanceQueue always advances under lock. UI has no wire:loading.attr="disabled". After complete-without-advancing, current is already completed; each tap advances past the next waiting serial.

Failure scenario: Fatima completed, still on card. Staff double-taps Call next. Rahim (#5) then Karim (#6) both advance; Rahim never announced; TV jumps #4→#6.

Solution: Make advance idempotent (no-op if current is still completed / next already called), OR require current ∈ {completed,null} and disable the button while loading.

L3. markAbsent cancels mid-consult; endSession does not

High

Where: LiveSessionService::endSession (~437–444) vs ::markAbsent (~520–526)

Problem: endSession completes in_chamber first, then cancels waiting/called/skipped. markAbsent cancels everything not completed/cancelled/no_show — including in_chamber.

Failure scenario: Fatima is in_chamber. Staff hits Cancel session (absent). Fatima becomes cancelled mid-consult — notes/prescription and ops stats disagree with End Session behaviour.

Solution: Align markAbsent with endSession: complete in_chamber, cancel only waiting/called/skipped.

L4. Outdoor screen midnight rollover drops a still-running late session

High

Where: ScreenController always uses Carbon::today(); screen.blade.php day-rollover reload

Problem: Stable TV URL resolves to today's date in APP_TIMEZONE. At midnight the page reloads to the new date's empty session.

Failure scenario: Evening clinic still calling #18 at 23:59. At 00:00 Dhaka the TV reloads to “today” with no live session while Live Queue Control may still be on yesterday's session_date.

Solution: Prefer the latest non-completed live session for that schedule, or keep serving yesterday until live_sessions.status is completed/cancelled.

L5. Vacation-mode vs booking race (check-then-act)

High

Where: BookingService block check then insert; SlotBlock::created cancel hook

Problem: Block check and booking insert are not atomic with SlotBlock creation.

Failure scenario: Patient booking TX locks session, sees not blocked. Admin creates SlotBlock (cancels existing). Patient insert commits → new waiting serial on a closed day; cancel hook already ran.

Solution: Re-check block after lock / under same lock as SlotBlock; or cancel-on-booking when block exists.

L6. Livewire Call now accepts any tenant booking ID (wrong session)

Medium

Where: LiveQueueControl::callPatientNow; LiveSessionService::callSpecificPatient

Problem: Booking::findOrFail is tenant-scoped (good vs cross-tenant) but callSpecificPatient never checks bookable_id / booking_date match the active live session.

Failure scenario: Authenticated queue runner crafts a Livewire call with another session's waiting booking ID → that patient is marked called on the wrong TV/session.

Solution: Require booking bookable_id/type/date to match the live session before setAsCurrent.

L7. Concurrent prescription saves last-writer-wins

Medium

Where: VisitRecordService::syncPrescription — items()->delete() then recreate

Problem: Delete-all + recreate with no row lock / version.

Failure scenario: Doctor mid-consult saves 3 medicines; staff Enter prescription saves 1 overlapping → doctor's other lines vanish.

Solution: Row-level lock on visit_record; optimistic versioning; or staff merge instead of replace when doctor fields exist.

L8. Old voice/photo deleted before successful DB update

Medium

Where: VisitRecordService::saveForCompletedBooking (~52–58)

Problem: deleteIfExists runs before updateOrCreate. If the write fails, clinical media is gone.

Failure scenario: Doctor replaces voice note; DB error after delete → previous recording unrecoverable.

Solution: Delete old files after successful commit (DB::afterCommit).

L9. SMS refund on gateway timeout may overstate wallet

Medium

Where: SmsService debit-before-send; any Throwable refunds

Problem: HTTP driver times out after gateway accepted the message → refund → free SMS.

Failure scenario: Busy Mark Late blast; timeouts on accepted sends → balance overstates.

Solution: Treat ambiguous timeouts as sent (no refund) or query delivery status; only refund on clear 4xx.

L10. Staff walk-ins bypass billing gate

Medium

Where: EnsureTenantAcceptsBookings only on public booking POST; BookingService does not call acceptsBookings()

Problem: Architecture claims BookingService enforces billing; service does not.

Failure scenario: Tenant past_due — public book blocked; desk staff still add walk-ins → new serials while Super Admin thinks bookings are closed.

Solution: Enforce acceptsBookings() inside createBookingForBookable (optional staff override flag if intentional).

L11. reinstatePatient has no status / session-state guards

Medium

Where: LiveSessionService::reinstatePatient

Problem: UI only shows Reinstate for no_show, but service will rewrite any booking to skipped with a new retry slot.

Failure scenario: Session already completed; crafted request reinstates a no-show → skipped forever with nobody calling.

Solution: Allow only no_show while live session is active/paused/delayed.

L12. Late-notice job: partial notify if process dies mid-loop

Medium

Where: SendDoctorLateNotices via ->afterResponse(); no checkpoint

Problem: Per-patient try/catch is good; no resume. No real queue worker.

Failure scenario: 30 waiting; PHP-FPM kills after-response after 12 SMS → 12 told, 18 not; staff think everyone was notified.

Solution: Persist late_sms_sent_at per booking; make job resumable; require a real queue worker with retries.

4. Security (medium / lower)

S8. Patient portal puts phone in query string

Medium

Where: GET portal?phone=01... — BookingController::portal

Problem: Exact match is good (not LIKE %), but PII lands in access logs, browser history, Referer. Hero booking was fixed to POST for this reason; portal was not.

Failure scenario: Shared clinic PC / proxy logs expose who looked up which number.

Solution: POST + PRG (same pattern as /book prefill), or one-time server-side token after POST.

S9. Log SMS driver logs full phone + message body

Medium

Where: LogSmsGateway; SMS_DRIVER=log default in .env.example

Problem: Every "sent" SMS (including prescription /p/{token} links) written to app log with destination and full body.

Failure scenario: Log access = patient phones + live prescription share URLs.

Solution: Never log full bodies/phones; redact to last-4 and purpose codes. Keep SMS_DRIVER=http as production blocker.

S10. Staff password rules are effectively non-empty only

Medium

Where: UserResource password TextInput — no min / Password::defaults()

Problem: Hashing via hashed cast is fine; strength rules missing.

Failure scenario: password=1 as chamber admin → easy guessing of a year-lived session.

Solution: Apply Laravel Password::defaults() on create/update.

S11. CSV export formula injection

Medium

Where: BackupCsv::serializeValue writes raw strings

Problem: Patient names / free text written raw into CSV (with BOM).

Failure scenario: Name =HYPERLINK("http://evil"&A1) — Excel/Sheets may execute formula when staff open the backup.

Solution: Prefix cells starting with =, +, -, @ with ' or a tab.

S12. Restore accepts any readable filesystem path from Livewire state

Medium

Where: DataBackup::restoreBackup — is_readable(\$path) only

Problem: Does not require path under the FileUpload disk/temp directory.

Failure scenario: Manipulated Livewire payload pointing at another readable ZIP on the server restores unintended data.

Solution: Resolve against configured upload disk root; reject absolute paths and ..

S13. Import bypasses Eloquent HTML sanitization (stored XSS)

Medium

Where: DataImportService DB upsert; clinic views {!! \$post->body !!}

Problem: Normal Filament saves run HtmlSanitizer; import writes body/bio via Query Builder.

Failure scenario: Malicious backup ZIP → stored script on public clinic pages.

Solution: Run HtmlSanitizer::clean on imported HTML columns, or sanitize again on output.

S14. Google Maps allowlist: any goo.gl link passes

Low-Medium

Where: Chamber::isGoogleMapsUrl — hosts maps.app.goo.gl, goo.gl, maps.google.com

Problem: Note: google.evil.com does NOT match the regex (verified). Real gap is bare goo.gl (shortener can redirect anywhere) and non-maps short links.

Failure scenario: Ticket/WhatsApp maps CTA sends patients off-site.

Solution: Drop bare goo.gl or resolve/validate destination; keep maps.app.goo.gl and maps.google.com / google.*/maps.

S15. AUTH_DEBUG / SessionProbe can log session IDs

Low

Where: AuthDebugProvider; SessionProbe; wired in bootstrap

Problem: No-ops unless AUTH_DEBUG=true. When on, logs truncated session IDs, user IDs, full URLs.

Failure scenario: Accidental enable in production expands sensitive log surface.

Solution: Remove from default boot path after the session bug is closed; never log URLs that may contain tokens.

S16. Demo seeders hardcode password "password"

Low

Where: NusratUrmiSeeder (and similar)

Problem: Known credentials if seeder runs on a reachable environment.

Failure scenario: Ops mistake → public known login.

Solution: Random passwords printed once, or refuse to seed when APP_ENV=production.

Verified as correctly handled (do not “fix” these)

- TenantScope throws on HTTP if tenancy is not initialized — no silent cross-tenant queries.
- BelongsToTenant forces tenant_id on create and blocks updates.
- Clinical HTTP routes check role + belongsToCurrentTenant().
- canAccessPanel() ties tenant panel access to matching tenant_id.
- Tenant DataBackup export uses tenant('id') only; files under storage/app/backup-temp, streamed then deleted — not under public/.
- Prescription share /p/{token}: 10-char base62, unique, expiry enforced, throttled, diagnosis off share view.
- Booking tickets use UUID route keys + TenantScope.
- Portal / by-phone: exact normalized BD phone; by-phone returns masked labels; throttled.
- Visit media: private local disk, UUID filenames, MIME allowlists, auth streaming.
- SafeUrl blocks javascript:, data:, protocol-relative //...
- Page-builder rich text sanitized on save and output.
- No custom CSRF \$except found; raw SQL usages parameterized.

5. Performance, polling, and database indexes

Polling map (current)

Surface	Interval	Approx. queries	Notes
Waiting-room TV	2s fetch	4–8	No cache; ETA re-fetches live session
Patient ticket	5s fetch	4–7	Same ETA cost
Live Queue Control	3s Livewire	4–6+	Duplicate bookings query + full day table
Consult Screen	3s wire:poll	5–10+	PHP-side visit history scan

At 20 clinics × 2 TVs × 30 polls/min ≈ 1,200 req/min from TVs alone. Zero Cache:: usage exists in app/. Session/cache/queue drivers default to database; no queue worker.

P1. No application caching; poll paths hit DB every time

High

Where: config/cache.php default database; grep of app/ → no Cache:: usage

Problem: Screen/ticket/consult/queue all hit DB every poll.

Failure scenario: Thousands of queries/min under modest multi-clinic load for screens that mostly show the same serial.

Solution: Cache screen + ticket JSON 1–2s keyed by live_session.id + updated_at. Prefer Redis for cache/session. Pass already-loaded live session into ETA (don't re-fetch).

P2. Live Queue / Consult duplicate & PHP-side scans every 3s

High

Where: LiveQueueControl getBookingsProperty + Filament table query; VisitRecordService lastRecordedVisitForPatient get()+PHP filter

Problem: Same day's bookings loaded twice; last visit / catch-up computed in PHP over full collections.

Failure scenario: Every open staff screen burns 5–10+ queries every 3 seconds.

Solution: One shared query; SQL EXISTS / ORDER BY LIMIT 1 for last visit; SQL for completed-without-notes; paginate or hide completed.

P3. Tailwind CDN on solo patient pages (BD 3G cost)

High (UX)

Where: tenant/solo/webpage, book, ticket, portal — cdn.tailwindcss.com

Problem: Runtime CDN is large; CDN failure = unstyled page. Clinic uses built clinic-clireo.css (better).

Failure scenario: Bangladesh 3G: homepage/book can lose 1–5+ seconds before interactive.

Solution: Build a CSS bundle for book/ticket/portal. Solo homepage markup is LOCKED — needs owner phrase update/change patient homepage before changing locked shells.

P4. Wrong-shape / missing indexes

Medium

Where: bookings_bookable_date_index; live_sessions unique; portal phone query; visit_records research

Problem: bookings_bookable_date_index = (bookable_type, bookable_id, booking_date) without tenant_id — redundant with bookings_serial_unique prefix. live_sessions queried by (tenant_id, session_date, status) but unique key starts with schedule_session_id. Portal whereIn(patient_phone) unindexed. CommissionService uses whereYear/whereMonth (defeats indexes).

Failure scenario: As bookings grow: table scans on portal and polls; extra write cost on redundant index.

Solution: Reshape or drop bookings_bookable_date_index to lead with tenant_id. Add live_sessions (tenant_id, session_date, status), bookings (tenant_id, patient_phone), visit_records (recorded_at). Replace whereYear/whereMonth with whereBetween.

P5. Medicine/condition catalogues loaded fully into PHP

Medium

Where: MedicineService::catalogForPracticeType get(); ConditionService get() then PHP match

Problem: Full catalogue (~460 medicines / ~244 conditions) into PHP for picker/search.

Failure scenario: Opening complete-visit form + typing diagnosis does unnecessary full scans.

Solution: Cache catalogue per practice type; SQL LIKE/prefix with LIMIT for search APIs.

P6. Sync SMS on booking request; DB session driver

Medium

Where: BookingService confirmation SMS after commit; config session/queue database

Problem: Booking request waits on SMS gateway; every Livewire poll hits sessions table.

Failure scenario: Slow book confirmation under gateway latency; DB contention under poll load.

Solution: Queue SMS; Redis sessions; keep afterResponse only as interim.

Already well-optimized (keep)

- DateOnly + plain where('booking_date') on roster/queue/booking service — index-friendly.
- HasManyByScheduleAndDate avoids broken eager whereDate across parents.
- Booking wizard sessionsPayload strips Eloquent noise.
- SellerOverviewService uses grouped aggregates, not per-tenant loops.
- TV announce WAVs load on demand (not all 99 preloaded).
- bookings_roster_index (tenant_id, booking_date, status) matches Daily Roster.
- Patients (tenant_id, phone) / (tenant_id, name); SMS (tenant_id, created_at|status).

6. Over-engineering and structural mess

Core app/Services (18) and app/Support (10) are mostly coherent one-job helpers. The mess is concentrated: dual tenant admin panels, URL helper mini-system, Filament god pages, solo/clinic view forks, and public/icons + docs clutter.

#	Issue	Simpler approach
OE1	Two Filament tenant panels (path + domain) for the same desk	Ship path-only until first paying custom-domain clinic; or one canonical panel + thin alias.
OE2	URL mini-system: TenancyUrl, FilamentPanelUrl, SafeUrl, helpers, publicAbsolute, screenBookmarkUrl	One cheat sheet: page link vs SMS/TV outbound. Stop adding modes. Keep path+domain need; shrink implementation.
OE3	God screens: LiveQueueControl ~954 + VisitNotesFormSchema ~777 + ConsultScreen ~400	Split when next touching UX: Queue actions / Visit notes form / Session lifecycle. Keep LiveSessionService as mutator.
OE4	Solo vs clinic = two websites (section trees duplicated)	Keep locked solo homepage pins. Unify book/ticket/portal shells + CSS tokens.
OE5	54 logo files in public/icons (~1.3 MB); app uses health-favicon.svg	Keep favicon + one logo; delete or move the rest out of deploy.
OE6	Settings knobs before use: 8 notify toggles, 3 ETA models, announce locale unused	Hard-code defaults for first clinics; add toggles only when a customer asks.
OE7	Full lab-tests vertical on by default for clinic	Default labs off until CBPH (or similar) needs them.
OE8	1,100-line booking-wizard.blade.php (HTML+JS+validation)	Extract JS to resources/js/booking-wizard.js; don't render dead steps.
OE9	Page builder with ~18 block types	Fixed template per tier + content fields unless doctors rearrange.
OE10	Sales PDFs/slides (~2.3 MB) + public/previews in the app repo	Move sales artifacts out of deploy path; keep deferred/stashes.
OE11	Three identical doctor can* methods; BookingController reimplements BdPhone	Collapse aliases; call BdPhone everywhere.
OE12	ROLE_PATIENT unused; Research aggregates before enough coded visits	Delete dead constant; soft-hide Research until MIN_GROUP_SIZE can fire.

Largest files (symptom list)

Lines	File	Job
~1105	booking-wizard.blade.php	Patient booking steps
~954	LiveQueueControl.php	Staff queue desk
~777	VisitNotesFormSchema.php	Prescription / notes modal
~695	LiveSessionService.php	Queue mutations (justified)
~627	consult-screen.blade.php	Doctor consult UI
~573	screen.blade.php	Waiting-room TV
~509	WebPageResource.php	Page builder
~500	live-queue-control.blade.php	Queue desk markup + CSS

7. Technology choices: shiny vs battle-tested

The stack is mostly mainstream Laravel 12 / Filament 4 — not experimental packages. The real risk is clever operational workarounds.

T1. No queue worker + afterResponse() for doctor-late SMS

Critical (ops)

Where: SendDoctorLateNotices; LiveSessionService::markDelay; QUEUE_CONNECTION=database

Problem: ShouldQueue job dispatched with ->afterResponse() so SMS runs without a worker.

Failure scenario: Loses retries, failed_jobs visibility, isolation. Gateway ~10s/patient × 30 can exceed max_execution_time mid-blast → partial notify.

Solution: Real queue worker (queue:work / Supervisor) + database/redis queue. Checkpoint late_sms_sent_at.

T2. Referrer-based tenancy for /livewire/update

Critical (ops)

Where: InitializeTenancyForTenantHosts — route → path → HTTP Referrer; no test coverage

Problem: Livewire POSTs to /livewire/update with no {tenant}. Referrer can be absent, stripped, or spoofed.

Failure scenario: Wrong-tenant context / CSRF 419 class of bugs (already in bug_history). Spoofed Referrer can initialize a different existing tenant slug.

Solution: Tenant-carrying Livewire update endpoint under /{tenant}/..., or persistent path middleware that always carries tenant. Do not rely on Referrer.

T3. Deep Filament fi-* CSS / Alpine internals coupling

High

Where: ConfiguresTenantAdminPanel; theme.css; VisitNotesFormSchema repeater-collapse / x-on:click.capture

Problem: Styles/behavior target Filament generated class names (.fi-collapsed, .fi-fo-repeater-item, builder classes) and internal Alpine events. Constraint is ^4.0 (any 4.x minor).

Failure scenario: Filament minor rename silently breaks visit-notes UX and page-builder buttons.

Solution: Pin Filament to ~4.12.0. Keep an upgrade checklist of fi-* / Alpine couplings. Prefer documented Filament APIs over capture-phase Alpine hacks.

T4. Hand-rolled CSV backup instead of mysqldump / Spatie

High (recovery)

Where: DataBackupService / DataImportService / BackupTableMap

Problem: Per-table CSV in ZIP + upsert restore. Passwords excluded/regenerated (good).

Failure scenario: Cannot restore clinical media; FK order fragile; schema drift silent; concurrent live traffic during restore unsafe. Looks like DR, isn't.

Solution: mysqldump or spatie/laravel-backup for bit-exact DB + filesystem off-box. Keep CSV as portability.

T5. CI PHP 8.4 vs production platform.php 8.3; symfony/html-sanitizer ^7 pin

Medium

Where: composer.json platform.php=8.3.0; .github/workflows/tests.yml PHP 8.4

Problem: CI can green packages that break on the VPS. Sanitizer pinned to ^7 to avoid Symfony 8 / PHP 8.4 Dom\HTMLDocument.

Failure scenario: Removing the pin on a newer laptop re-locks to Symfony 8 and breaks Filament on 8.3 (already happened).

Solution: Run CI on PHP 8.3 matching production; OR upgrade VPS to 8.4 and allow Symfony 8.

T6. Dual front-end: Vite Tailwind 4 + CDN Tailwind on solo

Medium

Where: vite.config.js; solo blades cdn.tailwindcss.com

Problem: CDN Tailwind ≠ built v4 utilities; past “classes silently do nothing” bugs.

Failure scenario: Patient UX depends on third-party CDN; admin/patient style divergence.

Solution: One compiled CSS pipeline everywhere (homepage lock applies to locked shells).

T7. Custom HasManyByScheduleAndDate relation

Medium

Where: `app/Models/Relations/HasManyByScheduleAndDate.php`; `LiveSession::bookings()`

Problem: Subclass `HasMany` for correct eager match across dates. Tested for eager match. Does not override `getRelationExistenceQuery` — `whereHas/withCount` may ignore date (no current usage).

Failure scenario: Eloquent upgrades can break custom relation matching; latent `whereHas` bug.

Solution: Document limitation; add existence-query override or load bookings with explicit `whereIn`.

T8. No static analysis / lint in CI

Medium

Where: `.github/workflows` — PHPUnit SQLite + MySQL only

Problem: No PHPStan/Larastan/Pint. Frontend lint none.

Failure scenario: Type/API mistakes and style drift land undetected.

Solution: Add Larastan + Pint to CI. Keep MySQL job — do not remove it to speed CI.

Correct / conservative technology choices

- minimum-stability: stable — no betas.
- Shared DB + app-level `BelongsToTenant` — right for modest VPS.
- `mews/purifier` for tenant-authored HTML.
- MySQL CI job — production-shaped.
- `GsmText` enforcement at `SmsService::send()` choke point.
- Prescription short token vs signed URL for SMS length.
- Private disk for clinical media + stream controllers.
- Carbon for dates; PHPUnit (not novelty frameworks).

8. Macro-architecture drift

Evidence of feature-by-feature growth without holding the whole system in view. Same jobs done more than one way; docs slightly drifted from code.

Same job, multiple ways

- **Phone:** Canonical BdPhone; BookingController reimplements normalize privately; PatientForm regex stricter (no +88) than booking.
- **URLs:** TenancyUrl / helpers / SafeUrl / FilamentPanelUrl / residual asset() / url() — easy to pick the wrong one.
- **Auth:** User::can* + 4 Policies + Filament canAccess + inline controller checks — not one choke point. Clinical routes need belongsToCurrentTenant (not middleware on all tenant routes).
- **Booking create:** CONFIRMED single path (BookingService). Status changes go through LiveSessionService / SlotBlockService — good.
- **HTML:** HtmlSanitizer on save; some views {!! !!} trusting stored HTML — import bypasses sanitizer.

Documentation vs reality (spot checks)

- CONFIRMED: BookingService single create path; GsmText choke point; DateOnly + no whereDate on hot path; STAFF_WRITABLE_FIELDS; visit media stream-only; catalogues required.
- DRIFTED: architecture claims “18 sections” — there are 17 under tenant/sections/.
- DRIFTED: sitemap.md still shows doctorgemini.com example host.
- INTERNAL DOC DRIFT: Folder Structure paragraph vs Key Components disagree about clinic Tailwind CDN status (Key Components is current — clinic has no CDN).
- DataBackup + Clinic CMS are documented in architecture/sitemap (not missing).

Top traps for a new maintainer

- 1. Touch solo homepage blades without unlock phrases (patient-homepage-lock).
- 2. Shorten SESSION_LIFETIME / restore framework 120 — locked (session-expiry-lock).
- 3. Add whereDate on booking_date / session_date — breaks indexes / SQLite vs MySQL story.
- 4. Pass Carbon into LiveSession::firstOrCreate WHERE — silent unique-key misses.
- 5. Normalize phone in only one place — BdPhone drift + PatientForm mismatch.
- 6. Rely on Policy alone for clinical HTTP — only 4 policies; real gate is can* + belongsToCurrentTenant.
- 7. Queue SMS / doctor-late as ShouldQueue without afterResponse or a worker — jobs never run today.
- 8. Put __() in SmsService bodies — Bangla multiplies GSM segments.
- 9. Add a public URL helper on visit media — defeats private-disk design.
- 10. Edit tenant/sections/* expecting solo to follow — solo pins copies under tenant/solo/sections/.

9. What is correctly built (leave alone)

Do not rip these out in the name of “simplicity.” They earned their keep from real bugs or privacy requirements.

Area	Why it stays
Serial allocation	DB::transaction + lockForUpdate on bookable; max()+1 inside TX; unique (tenant_id, bookable_type, bookable_id, booking_date, serial_number).
LiveSession mutations	Nearly every mutation uses transaction + lockSession().
SMS wallet	Atomic WHERE sms_balance >= credits + decrement; cannot go negative; refund on hard failure.
Monthly commissions	firstOrCreate on (marketer_id, tenant_id, type, period) with non-null Y-m; re-run safe.
DateOnly cast	Writes Y-m-d so SQLite/MySQL date equality and indexes stay honest.
Clinical media privacy	Private local disk; streamResponse only; MIME allowlists; no public URL accessor.
GsmText	Every SMS body forced through toSingleSegment() inside SmsService::send().
Prescription short link	/p/{token} one GSM segment; signed long URL was correctly replaced.
TenantScope	Fails closed when tenancy not initialized.
Production readiness	Machine gate for silent misconfig (debug, log mail/SMS, empty catalogues).
Medicine/condition catalogues	Real BD prescribing UX — not decorative.
Path + domain tenancy (need)	BD deploy reality. Shrink implementation; keep the need.

10. Phased remediation plan (recommended order)

Ordered by what can cause unrecoverable harm first. Each phase is independently shippable. After each phase: full test suite + `php artisan app:production-check` on MySQL.

Phase 1 — Stop the backup system from causing damage (DO FIRST)

The restore feature is currently more dangerous than having no restore feature.

- Disable tenant-facing restore in TenantAdmin DataBackup until Phase 2 lands. Keep export/download (safe).
- Never upsert on bare id. Remap IDs or refuse cross-tenant / null-tenant (Super Admin) collisions.
- Wrap wipe + import in one transaction so a failed restore rolls back.
- Fix table ordering: `live_sessions` after `bookings`; null `current_booking_id` before wipe.
- Validate ZIP entry names before `extractTo` — reject `..` and absolute paths; allowlist only.
- Escape CSV formula characters on export (`=`, `+`, `-`, `@`).
- Tests: restore with active `current_booking_id`; ZIP with `foreign users.id`; mid-import failure rolls back. Run on MySQL.

Phase 2 — Real disaster recovery

The CSV tool does not back up clinical media, so it cannot restore a clinic.

- Add `mysqldump` (or `spatie/laravel-backup`) plus `visit-audio/` and `visit-photos/`, written off-server.
- Keep CSV export as per-chamber data portability; relabel UI so nobody mistakes it for DR.

Phase 3 — Queue and consult correctness

Logic bugs that lose patients in the queue or erase clinical data.

- Record medicine/condition usage: complete booking before save notes, or force usage after status flip.
- Make `Call` next idempotent + `wire:loading.attr="disabled"`.
- Align `markAbsent` with `endSession` (complete in `_chamber`, don't cancel).
- Guard `reinstatePatient` to `no_show` only while session live.
- Protect concurrent prescription saves (lock / merge).
- Delete replaced media after commit (`DB::afterCommit`).
- Do not refund SMS credits on gateway timeout — only clear rejection.
- Enforce `acceptsBookings()` inside `BookingService`.
- Honour `published_at` in `scopePublished`; add tenants FK to `departments/blog_posts`.

Phase 4 — Security configuration

Config and small code fixes. Does NOT unlock or change session lifetime.

- `SESSION_SECURE_COOKIE=true` and `SESSION_ENCRYPT=true`; promote Secure to ProductionReadiness blocker in production.
- Narrow `trustProxies` to real proxy IP(s).
- `Password::defaults()` on staff password fields.
- Redact SMS logging (no full phone/body).
- Portal phone lookup via POST, not query string.
- Restrict restore file path to upload disk.
- Reject non-Maps `goo.gl` links.
- Remove `AuthDebugProvider` / `SessionProbe` from default boot.

Phase 5 — Performance and indexing

Cut poll query cost and fix index shape.

- Cache TV/ticket poll payloads 1–2s; stop ETA double-fetch.

- Remove duplicate Live Queue query; replace PHP history scans with SQL.
- Reshape/drop bookings_bookable_date_index; add live_sessions (tenant_id, session_date, status), bookings (tenant_id, patient_phone), visit_records (recorded_at).
- Replace whereYear/whereMonth in CommissionService with whereBetween.
- Replace CDN Tailwind on book/ticket/portal (solo homepage needs unlock phrase).

Phase 6 — Reduce fragility

Upgrade safety and ops maturity.

- Pin Filament to ~4.12.0; maintain fi-* / Alpine upgrade checklist.
- Move CI to PHP 8.3 to match production; add PHPStan/Larastan + Pint.
- Run a real queue worker for SMS; drop Referer tenancy fallback for a tenant-carrying route.
- Fix doc drift: 17 not 18 sections; doctorgemini.com in sitemap; architecture Tailwind CDN contradiction.

11. Verification checklist for the developer

- All new/changed behaviour covered by Feature tests (especially backup on MySQL with active `live_sessions.current_booking_id`).
- php artisan test on SQLite AND MySQL (CI already has both — keep both).
- php artisan app:production-check --strict on a production-shaped .env.
- Manual: Mark Late with many waiting patients — confirm all receive SMS or resumable checkpoint.
- Manual: Complete visit with medicines — confirm `medicine_usages` / `condition_usages` increment.
- Manual: Double-tap Call next — confirm only one patient advances.
- Manual: Cancel session while patient in_chamber — confirm completed not cancelled.
- Manual: Restore a chamber ZIP that reuses another user's id — must refuse or remap, never overwrite Super Admin.
- Confirm `SESSION_SECURE_COOKIE` and `SESSION_ENCRYPT` set; session lifetime still 525600 (do not change).
- Confirm solo homepage blades untouched unless owner said update/change patient homepage.
- Update `decisions.md` / `bug_history.md` / `architecture.md` / `architecture_history.md` / `sitemap.md` per `AGENTS.md` before declaring done.

12. Owner constraints the developer must not violate

These are explicit owner decisions. “Harden the app”, “fix security”, or a general cleanup pass does **not** unlock them.

Session expiry lock

Staff stay signed in indefinitely on purpose. `SESSION_LIFETIME=525600` (one year) in .env, .env.example, **and** the default in `config/session.php`. `SESSION_EXPIRE_ON_CLOSE=false`. `AUTH_PASSWORD_TIMEOUT=31536000`. Do not restore Laravel's 120-minute default. Unlock phrases: **change session expiry** or **restore session timeout**. You MAY set Secure/Encrypt cookies without unlocking.

Patient homepage lock

Do not change `resources/views/tenant/solo/webpage.blade.php` or `resources/views/tenant/solo/sections/**` or homepage Book Appointment CTA wiring/styling unless the owner says **update patient homepage** or **change patient homepage**. Book CTAs must keep `tenant_safe_href(..., '/book')`. Book/ticket/portal shells outside the locked homepage may be improved (e.g. replace CDN Tailwind) without that phrase.

Other product constraints

- No online payment — pay-at-chamber until the owner asks.
- SolDoc is pre-production (not a prototype) as of 2026-08-08 — security, backups, and correctness are in scope.
- Do not remove the MySQL CI job to speed things up.
- Project memory (`decisions.md`, `bug_history.md`, `architecture.md`, `architecture_history.md`, `sitemap.md`) must be updated when behaviour or structure changes — see `AGENTS.md`.

End of brief. Fix Phase 1 before anything cosmetic. Questions about locked decisions must go to the owner — do not silently override them.

Generated 2026-08-11 03:10 · ChamberQ codebase audit for developer handoff